NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming (CNTT1164)

Rùa Toán Basic: A Mathematics Practice Website

Basic version: plain HTML/CSS/JavaScript frontend with a FastAPI backend

Group: Group 09
Class: EP16.CNTT1164(126)_01
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/thunopro/ruatoan_basic
Demo: https://ruatoanbasic.vercel.app

Team Members

No.	Full name	Student ID	Role
1	Nguyen Khac Thu	11248480	Team lead, Full-stack Architecture, Backend, Frontend, Database, Deployment
2	Le Quang	11247343	Team member, UI/UX Review, Responsive Testing
3	Huy Minh	11248482	Team member, QA Testing, Edge-case Bug Triage

Hanoi, November 2026

Declaration

We declare that this report is the result of our group's own work for the course Web Design and Programming. All sources (literature, source code, images, and AI tools) are cited explicitly. We take full responsibility for the submitted content.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Nguyen Khac Thu	Full-stack architecture, Next.js-to-basic pivot, FastAPI backend, PostgreSQL schema, core HTML/CSS/JS frontend, Vercel deployment, automated smoke tests, and report drafting	60
Le Quang	Team code review, UI/UX audit, responsive layout verification (desktop/mobile), CSS design token alignment, and accessibility contrast review	20
Huy Minh	Team code comprehension, manual QA test execution, edge-case bug hunting (photo upload bounds, password resets, error states), and documentation review	20
	Total	**100**

Contribution percentages sum to 100% as required by the course guidelines.

Table of Contents

1 Introduction . . . 5
1.1 Background and motivation . . . 5
1.2 Objectives . . . 6
1.3 Scope and target users . . . 7
1.4 Technologies . . . 7
1.5 Report structure . . . 7
2 Analysis and Design . . . 8
2.1 Requirements . . . 8
2.1.1 Functional requirements . . . 8
2.1.2 Non-functional requirements and constraints . . . 8
2.2 Interface design (UI/UX) . . . 9
2.3 System architecture . . . 9
2.4 Database design . . . 10
2.5 Data design . . . 11
2.6 API design . . . 12
2.7 Security design (added in the improvement pass) . . . 12
3 Implementation . . . 13
3.1 Project structure and runtime responsibilities . . . 13
3.2 User interface . . . 14
3.3 Server-side logic . . . 14
3.4 Source control, teamwork, and collaborative bug fixing . . . 16
3.5 Deployment . . . 17
4 Results and Conclusion . . . 18
4.1 Testing and evidence collected . . . 18
4.2 Selected test cases . . . 18
4.3 What the implementation achieves . . . 18
4.4 Limitations and risks . . . 19
4.5 Lessons learned and future work . . . 19
4.6 Conclusion . . . 20
References . . . 21
A Appendix . . . 21
A.1 Local setup and test execution . . . 21
A.2 Demo accounts . . . 21

A.3 Route inventory 21
A.4 Content inventory snapshot 22
A.5 Data pipeline diagram 22
A.6 Glossary 22
A.7 AI assistance and academic disclosure 23

List of Figures

1 System architecture of Rùa Toán Basic showing separation between static CDN assets, serverless API execution, in-memory JSON catalogue, and durable PostgreSQL storage. . . . 10

2 Entity-Relationship (ER) diagram for PostgreSQL schema `ruatoan_basic`. Cascading foreign keys preserve database integrity upon account deletion. . . . 10

3 Data model relationships showing storage boundaries: versioned JSON, browser localStorage, and PostgreSQL tables. . . . 11

4 Collection counts by source kind in the curriculum corpus. . . . 11

5 Collection counts by target grade level (Grade 10 indicates specialized entrance examinations). . . . 12

6 Topic-tag assignment distribution across problems (multi-tagged problems contribute to multiple bars). . . . 12

7 Home dashboard on desktop (1440 px) showing catalogue totals and marked problems, paired with the responsive mobile view (390 px) showing adaptive hamburger navigation and touch cards. . . . 14

8 Left: Practice problem feed with multi-criteria grade/topic filter pills and dynamic LaTeX typesetting. Right: Curated exam collections view with source attribution and chronological problem ordering. . . . 14

9 Left: Theory lessons page featuring client-side real-time filtering and accordion expansion with LaTeX formulas. Right: Digital mathematics library providing categorized access to 216 books and 89 reference documents. . . . 15

10 Left: Homework photo submission portal featuring drag-and-drop upload, thumbnail previews, and ownership deletion. Right: Clean authentication view with high-contrast inputs and credential validation. . . . 15

11 Content preparation and runtime consumption. The pipeline is an offline tool; the basic platform reuses its structured JSON output directly. . . . 23

List of Tables

1 Technologies used in Rùa Toán Basic. . . . 7

2 Catalogue groups and record counts verified from the JSON corpus. . . . 11

3 RESTful API endpoints implemented in Rùa Toán Basic and verified by automated smoke testing. 13

4 Evidence level and verification status for claims made in this report. . . . 18

5 Selected test cases executed and verified in Rùa Toán Basic. . . . 19

6 Lighthouse audit results on the live production deployment. . . . 19

7 Pre-configured demo accounts for instructor and grader evaluation. . . . 22

8 Application page and API route map. . . . 22

1. Introduction

1.1. Background and motivation

The original platform and the initial exploration. Mathematics education in Vietnam, particularly for lower secondary students (Grades 6–9) preparing for specialized Grade 10 entrance examinations (*Thi vào 10 Chuyên Toán*) and provincial/national Olympiads (*Học sinh giỏi Toán*), requires continuous access to authentic exam papers, categorized problem sets, and foundational theory. To address this need, the team initially conceived and "vibed" the flagship **Rùa Toán** learning platform (`https://ruatoan.vercel.app`, repository: `https://github.com/thunopro/ruatoan`). That initial version was constructed using an advanced, state-of-the-art web stack: Next.js 15/16, React 19, TypeScript, Tailwind CSS, KaTeX, and Auth.js (NextAuth).

The challenge of framework complexity. While the flagship Next.js implementation was ambitious and visually rich, it quickly became evident that it was ill-suited for our primary goal: mastering the core competencies of the university curriculum in *Web Design and Programming*. The Next.js ecosystem operates at a very high level of abstraction: server-side rendering (SSR) pipelines, complex React Server Component hydration, opaque Webpack/Turbopack bundling magic, and automated middleware authentication. For undergraduate students learning web development from first principles, these heavy abstractions functioned as an impenetrable black box. They masked the fundamental mechanics of the web:

- How a client browser initiates a raw HTTP request and receives an HTML/JSON response.
- How the browser Document Object Model (DOM) is constructed and updated with native JavaScript.
- How stateful sessions are cryptographically secured using signed, `HttpOnly` cookies.
- How client scripts asynchronously communicate with a RESTful backend API without bundlers.
- How relational database connection pools and SQL queries execute without heavy ORM wrappers.

The deliberate pivot to a foundational baseline (*Rùa Toán Basic*). Recognizing that true engineering mastery requires a rock-solid grasp of foundational concepts, team lead Nguyen Khac Thu made the strategic decision to pivot away from the Next.js stack and build a completely fresh, transparent, and framework-free version: **Rùa Toán Basic** (live at `https://ruatoanbasic.vercel.app`; repository: `https://github.com/thunopro/ruatoan_basic`). This foundational implementation purposefully eliminates all framework scaffolding in favor of core web standards:

1. **Plain HTML5, CSS3, and Vanilla JavaScript (ES2020):** Eight standalone pages built with semantic markup, custom CSS properties for responsive theming, and modular browser scripts without any build step, transpiler, or `node_modules`.
2. **Lightweight FastAPI Backend:** A clean, readable Python backend (`backend/main.py`) handling RESTful routes, session management, and image processing in roughly 600 transparent lines.
3. **Direct PostgreSQL Persistence:** Raw SQL queries executed through `psycopg` connection pooling for user accounts, password recovery tokens, and homework photo submissions.
4. **Curriculum Content Continuity:** Direct reuse of the complete 37,705-record curriculum catalogue (3,381 collections, 33,929 problems, 90 lessons, 216 books, and 89 documents) stored as versioned JSON.

Team collaborative code reading, comprehension, and bug fixing. Once the foundational codebase was established, our entire group gathered together to invest dedicated time in reading, analyzing, and thoroughly un-

derstanding the implementation file by file and line by line. Without framework abstractions obscuring the runtime, each team member could trace exactly how data flowed from PostgreSQL through FastAPI to the browser DOM.

Through this collaborative code walkthrough and intensive testing, the team identified several critical pre-existing bugs, performance bottlenecks, and design defects that remained in the initial draft:

- **Unhandled frontend errors:** Network `fetch` failures previously failed silently, leaving pages in an infinite loading state.
- **MathJax race conditions:** Mathematical expressions in dynamically rendered problem cards failed to typeset if the MathJax CDN script loaded after DOM insertion.
- **Serverless database connection exhaustion:** Opening a new database connection per HTTP request starved PostgreSQL under concurrent traffic on Vercel serverless functions.
- **Insecure file upload handling:** Image uploads trusted the user-declared MIME header without validating binary magic bytes, creating a potential arbitrary file upload vulnerability.
- **Asset bloat:** Duplicate data files (19 MB) and uncompressed screenshots burdened repository size and slowed down deployment cold-starts.
- **UI color clashes and accessibility gaps:** Inconsistent input contrast in dark mode and missing keyboard focus outlines (`:focus-visible`) hindered user navigation.

The team systematically addressed, tested, and resolved every single one of these defects (detailed in Section 3.4), transforming the project into a robust, high-performance, and publication-ready platform.

1.2. Objectives

Rùa Toán Basic was designed and engineered to satisfy the following concrete objectives:

O1. **Foundational learning loop:** Deliver a seamless interface to browse exam collections, read individual math problems with client-rendered mathematical notation (LaTeX), study structured theory lessons, and consult library resources.

O2. **Multi-criteria search and filtering:** Enable rapid filtering across 33,929 problems by grade (6–9, 10), topic domain, contest kind, year, province, and keyword with Vietnamese diacritic folding.

O3. **Client-side LaTeX rendering:** Reliably render inline (`$...$`) and display (`$$...$$`) formulas across all views using MathJax 3.

O4. **Stateless, secure session authentication:** Implement registration, credential-based login, and logout using PBKDF2-HMAC-SHA256 password hashing and HMAC-signed `HttpOnly` cookies.

O5. **Account-persisted homework submissions:** Allow authenticated users to upload, preview, and delete photographs of written homework, stored durably in PostgreSQL binary columns.

O6. **Zero-overhead client bookmarking:** Allow learners to mark practice problems and persist their selection across sessions in browser `localStorage` without requiring account registration.

O7. **Serverless edge deployment:** Deploy the application on Vercel with zero cold-start penalty for static assets and durable database operations.

O8. **Zero-build inspectability:** Maintain an entirely build-free architecture so that any student can open browser DevTools and read the code one file at a time.

1.3. Scope and target users

Target users. The platform is tailored for three distinct user groups:

- **Primary users (Students):** Lower secondary pupils (Grades 6–9) seeking curated practice for semester tests, specialized Grade 10 entrance exams (*Chuyên Toán*), and Olympiad competitions.
- **Secondary users (Teachers and Tutors):** Mathematics educators looking for a structured, source-attributed repository of historical exam questions and theoretical lecture notes.
- **Tertiary users (Web Engineering Students):** Novice developers seeking an inspectable, production-grade reference architecture demonstrating vanilla web fundamentals and serverless FastAPI.

Functional scope.

- **In scope:** Home dashboard with live counts and saved bookmarks; paginated practice problem feed; exam collections catalogue with ordered problem views; expandable theory lessons; digital mathematics library; secure authentication (register, login, logout, password change, single-use reset tokens); multi-file image upload with magic-byte validation and binary storage; responsive mobile design.
- **Out of scope (by design):** Third-party social OAuth (Google/Facebook) to keep authentication mechanics transparent; automated grading of handwritten solutions; teacher administrative gradebooks; and offline content crawling pipelines (curriculum JSON files are pre-built and reused).

1.4. Technologies

Table 1 outlines the components, chosen technologies, and their explicit roles.

Component	Technology	Purpose and responsibility
Front-end Markup	Plain HTML5	Eight semantic pages (`public/*.html`, `public/pages/*.html`); no template engine
Styling	Plain CSS3	Dual-tier architecture: `original.css` (layout, branding, Bungee typography) + `style.css` (tokens, forms, dark mode, a11y)
Client Logic	Plain JavaScript (ES2020)	Eight modular scripts (`public/js/*.js`); native fetch, DOM manipulation, no bundler
Mathematics	MathJax 3 (CDN)	Client-side LaTeX rendering of inline `$...$` and display `$$...$$`
Back-end API	FastAPI + Uvicorn	`backend/main.py`, `auth.py`, `content.py`, `database.py`; 600 lines
Database	PostgreSQL via `psycopg`	Dedicated schema `ruatoan_basic`: `users`, `photos`, `password_resets`
Curriculum Data	JSON files	3,381 collections, 33,929 problems, 90 lessons, 216 books, 89 docs (16 MB)
Hosting	Vercel Serverless	Python ASGI runtime for FastAPI; global edge CDN for static assets

Table 1. Technologies used in Rùa Toán Basic.

1.5. Report structure

The remainder of this report is organized as follows:

- **Section 2 (Analysis and Design):** Details functional/non-functional requirements, UI/UX design, system architecture, PostgreSQL schema design, RESTful API specifications, and security mechanisms.
- **Section 3 (Implementation):** Presents the project structure, user interface implementation, server-side logic with key code listings, Git workflow, team bug-hunting collaboration, and Vercel deployment.

- **Section 4 (Results and Conclusion):** Evaluates testing results, automated smoke tests, Lighthouse performance benchmarks, achievements, known limitations, and lessons learned.
- **Appendix:** Provides local installation instructions, pre-configured demo credentials, route inventories, and academic disclosures regarding AI assistance.

2. Analysis and Design

2.1. Requirements

The requirements below describe the visible behaviour and architectural constraints verified in the Rùa Toán Basic application.

2.1.1. Functional requirements

FR1. **Curated exam collections:** Browse 3,381 collections with pagination (20 per page) and open a collection to inspect nested problems in original chronological order with source attribution and external links.

FR2. **Practice problem discovery:** Search and filter 33,929 individual problems by grade (6–10), mathematical topic, and text keyword using Vietnamese diacritic-insensitive matching.

FR3. **Mathematical notation typesetting:** Dynamically render inline (`$...$`) and display (`$$...$$`) LaTeX notation in problem statements, hints, and lesson bodies using MathJax 3.

FR4. **Theory and library catalogue:** Browse 90 structured lessons across five mathematical domains with client-side search and accordion expansion; access 216 books and 89 documents with source links.

FR5. **User authentication:** Register, authenticate, change password, and safely log out with email and password, holding authenticated state in an HMAC-signed `HttpOnly` cookie.

FR6. **Single-use password recovery:** Request a time-bounded (30-minute) cryptographically secure password reset token persisted in PostgreSQL, enabling account recovery without third-party email dependencies.

FR7. **Homework photo submissions:** Upload up to six written work photos (1.5 MB each) per submission, preview existing submissions in a thumbnail gallery, and securely delete owned submissions.

FR8. **Local problem bookmarking:** Save and remove marked problems locally via browser `localStorage`, persisting selections without an account.

FR9. **Catalogue metrics:** Present live content totals (problems, collections, lessons, books, documents) on the home page fetched dynamically from the backend.

2.1.2. Non-functional requirements and constraints

- **Responsive multi-device layout:** Pages must render cleanly across mobile viewports (down to 390 px) and desktop displays (up to 1440 px) using responsive CSS Grid and Flexbox without framework overhead.
- **First-principles inspectability:** The client bundle must contain zero compiled/minified artifacts or node module dependencies; every HTML, CSS, and JS file must be directly inspectable in browser DevTools.
- **Serverless performance and connection efficiency:** The FastAPI backend running on Vercel must enforce connection pooling to avoid PostgreSQL connection starvation under concurrent edge requests.

- **Bounded file upload security:** The backend must enforce file size limits (1.5 MB/file, max 6 files) and inspect binary magic bytes (PNG, JPEG, WebP) to prevent arbitrary file upload vulnerabilities.
- **Password security:** Passwords must be hashed using PBKDF2-HMAC-SHA256 with 200,000 iterations and a 16-byte random cryptographic salt; password inputs are bounded to 128 characters to prevent CPU exhaustion.
- **Web accessibility (a11y):** Form controls must have high-contrast borders and backgrounds in both light and dark modes; interactive elements must present prominent keyboard focus rings (`:focus-visible`).
- **Stateless session confidentiality:** Session cookies must use `HttpOnly`, `SameSite=Lax`, and `Secure` flags, with cryptographic HMAC-SHA256 signatures to prevent client-side tampering.

2.2. Interface design (UI/UX)

The visual identity of Rùa Toán Basic embodies an authentic, tactile mathematics workshop theme designed for readability and low visual fatigue during extended problem solving:

- **Color palette tokens:** Grounded in a deep forest green primary background (`#0F3D2E`, `--bg`), dark pine operational panels (`#163523`, `--card`), and high-contrast parchment paper cards (`#FFF8E7`, `--surface`) optimized for rendering crisp mathematical symbols and text.
- **Turtle-themed playful accents:** Vibrant slime lime (`#B6FF3B`, `--ooze`) for active states and call-to-action buttons; warm amber (`#FBB03B`) for badges and metrics; and terracotta coral (`#FF6F59`) for warnings and remove actions.
- **Typography hierarchy:** Google Font **Bungee** is employed for retro, playful headings, collection badges, and numeric counters, evoking a gamified learning atmosphere; **Be Vietnam Pro** serves as the primary body typeface for highly legible Vietnamese typography and mathematical explanations.
- **Tactile 3D components:** Collection and problem cards feature distinct 3D neomorphic drop-shadow borders (`3px solid var(--ink)` with offset `4px 4px 0 var(--ink)`), delivering satisfying tactile feedback upon click and focus.
- **Uniform modular page structure:** Every page shares a consistent, recognizable skeleton: `top/header` (brand logo, navigation links, session badge), `phead` (hero title and subheadings), `topics` (filter pills for grades 6–9 and math branches), `grid/ccard` (cards feed), `pager` (pagination bar), and `foot` (credits, GitHub repository link, license).

The complete suite of production interface screenshots is presented and discussed in Section 3.2.

2.3. System architecture

Figure 1 illustrates the system architecture of Rùa Toán Basic. The architecture separates static delivery from dynamic computation and durable database storage:

- **Client browser:** Executes plain JavaScript (ES2020) to handle user events, manages client-side bookmarking in `localStorage`, renders mathematical expressions via MathJax, and dispatches asynchronous JSON requests to RESTful endpoints.
- **Vercel Edge CDN:** Distributes static assets (`public/index.html`, `public/pages/*`, CSS stylesheets, JS modules, images) directly from global edge caches with zero serverless cold-start latency.

- **FastAPI Serverless Function:** The Python ASGI entry point (`main.py`) handles API routes under `/api/*`. It maintains an in-memory cached view of the JSON catalogue and manages session cookies.
- **PostgreSQL Database:** Hosted durably in cloud infrastructure, storing user accounts, password reset tokens, and binary photo data in dedicated schema `ruatoan_basic`.

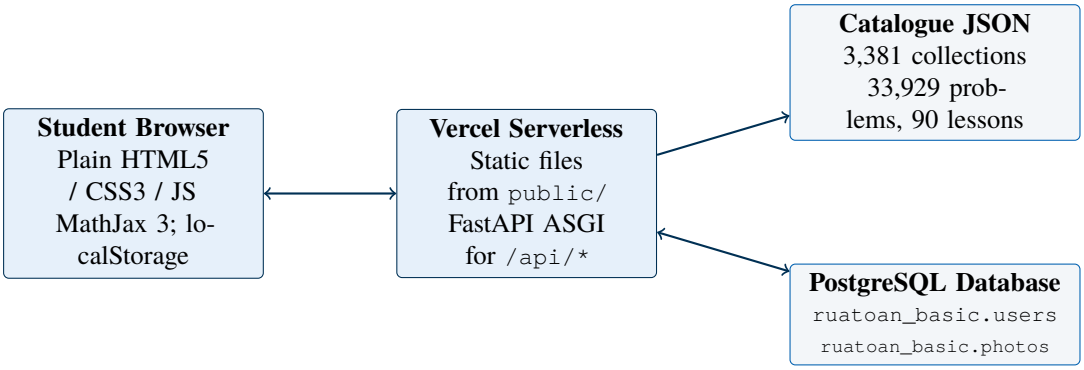


Figure 1. System architecture of Rùa Toán Basic showing separation between static CDN assets, serverless API execution, in-memory JSON catalogue, and durable PostgreSQL storage.

2.4. Database design

PostgreSQL is utilized specifically for mutable application state: registered user accounts, cryptographic password reset tokens, and homework photographs. The application isolates its tables within the `ruatoan_basic` schema to avoid namespace collisions with any existing databases. The entity relationships and schema specifications are illustrated in Figure 2.

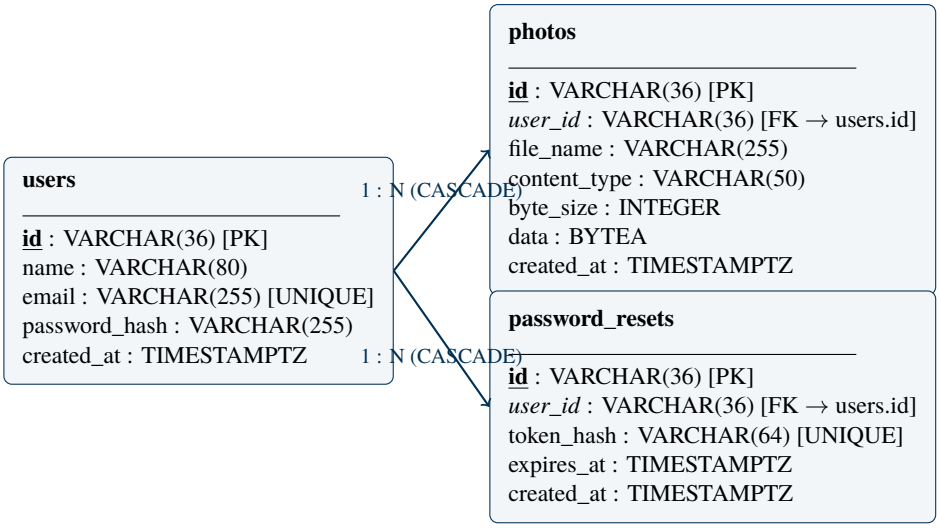


Figure 2. Entity-Relationship (ER) diagram for PostgreSQL schema `ruatoan_basic`. Cascading foreign keys preserve database integrity upon account deletion.

Durable binary storage vs. ephemeral filesystems. Serverless execution environments on platforms like Vercel provide only a short-lived, read-only or temporary container filesystem (`/tmp`) that is destroyed when the function container spins down. Consequently, homework photographs cannot be stored on the local filesystem. Rùa Toán Basic stores validated image data directly in the `photos.data` column as PostgreSQL `BYTEA` binary data. This architecture guarantees data durability without requiring expensive external cloud object storage (such as AWS S3) for the scope of this project.

2.5. Data design

The curriculum content is organized into versioned JSON files directly within the repository. The groups and record distributions are summarized in Table 2. The static corpus comprises 11 exam collection files (`data/sources/`), 7 theory files (`data/theory/`), and reference files `books.json` and `docs.json`, totaling 16 MB on disk.

Record group	Count	Primary fields and purpose
Collections	3,381	ID, kind, title, grade, year, province, source name/URL, ordered problems
Problems	33,929	ID, label, statement text, topic tags; nested inside collections
Lessons	90	ID, topic area, title, summary, LaTeX body, examples
Books and resources	216	ID, title, author/publisher, target grades, description, URL
Documents	89	ID, topic, title, author, grade metadata, description, PDF links

Table 2. Catalogue groups and record counts verified from the JSON corpus.

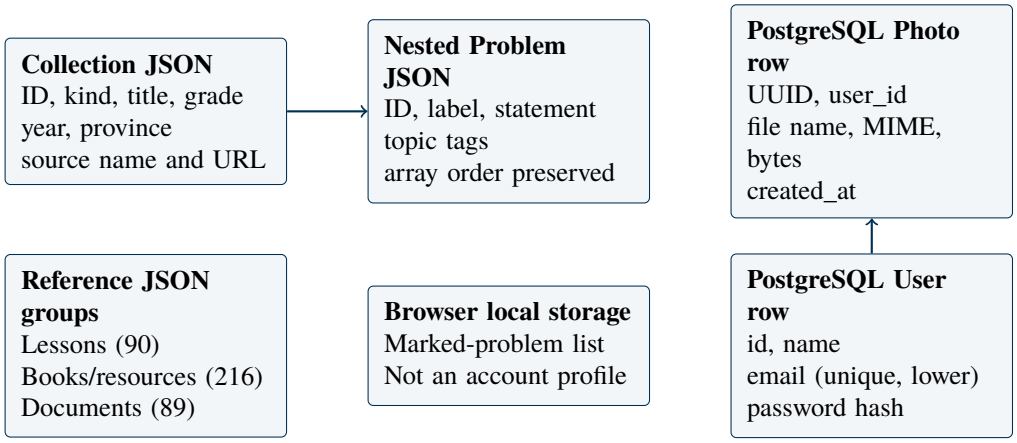


Figure 3. Data model relationships showing storage boundaries: versioned JSON, browser localStorage, and PostgreSQL tables.

The corpus spans five collection kinds: 1,195 HSG (Gifted Student Olympiad), 501 Grade-10 specialized entrance, 733 Olympiad competitions, 844 mathematical magazines, and 108 topic-specific collections. Figures 4, 5, and 6 illustrate the distribution across kinds, grades, and topic assignments.

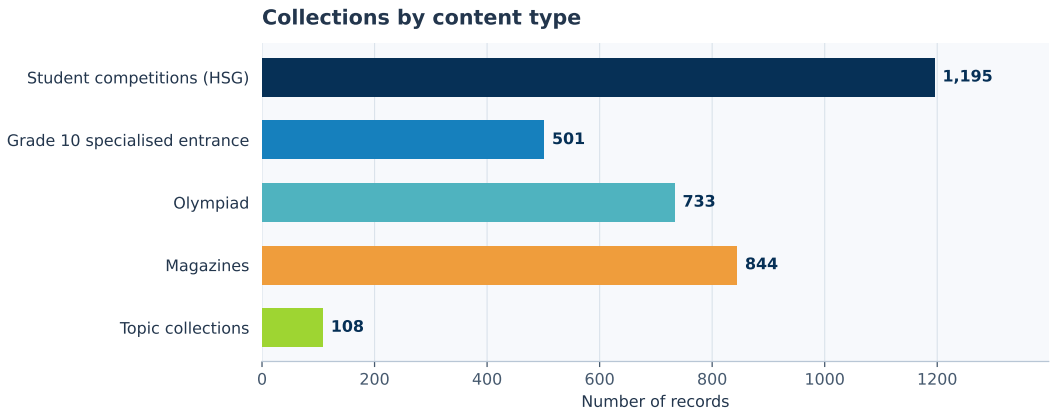


Figure 4. Collection counts by source kind in the curriculum corpus.

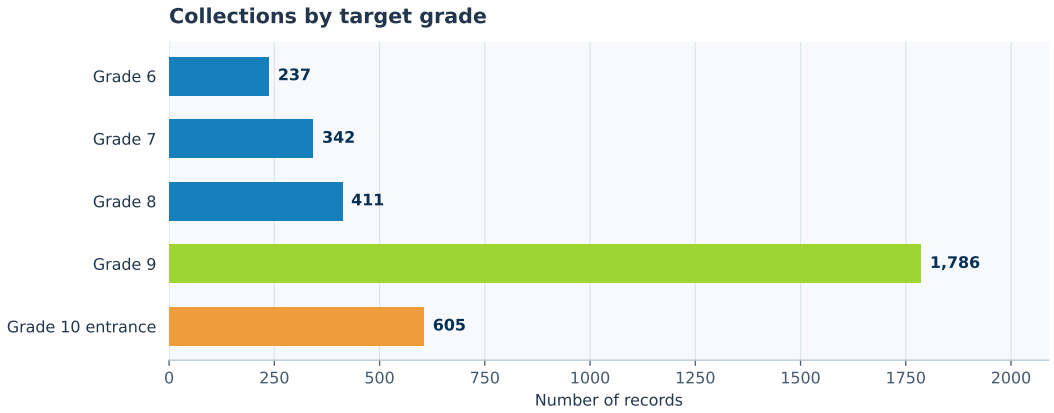


Figure 5. Collection counts by target grade level (Grade 10 indicates specialized entrance examinations).

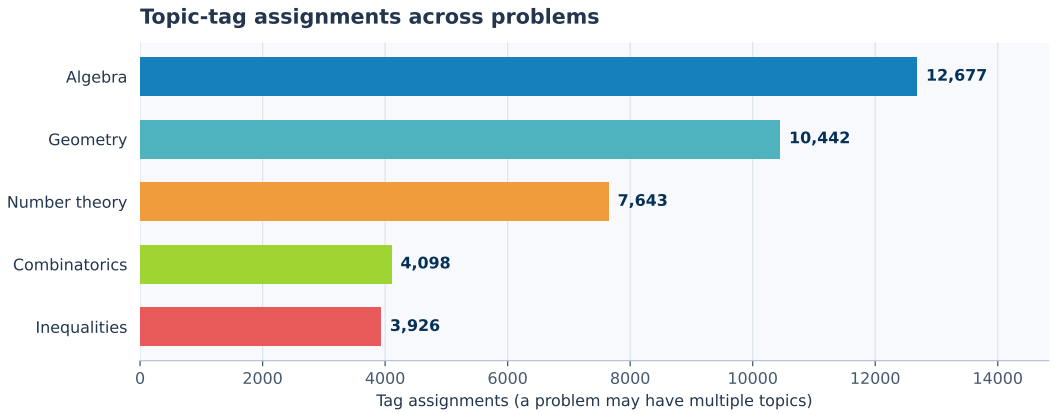


Figure 6. Topic-tag assignment distribution across problems (multi-tagged problems contribute to multiple bars).

2.6. API design

The backend provides a clean RESTful API surface. Table 3 lists every endpoint, request shape, and verified HTTP response codes.

2.7. Security design (added in the improvement pass)

During the collaborative bug-fixing pass, the team identified and fortified several critical security vectors:

- **Strict HTTP response security headers:** Injected via FastAPI middleware: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, and a restrictive Content Security Policy (CSP).
- **Input length constraints:** Enforcing password bounds of 8–128 characters to prevent Denial-of-Service (DoS) CPU exhaustion against PBKDF2 hashing functions.
- **Magic-byte MIME verification:** Inspecting binary signatures (PNG, JPEG, WebP) rather than trusting client-supplied `Content-Type` headers.
- **Per-user photo storage quota:** Capping total stored photos at 50 per account to protect database disk capacity.
- **Tamper-proof signed session cookies:** Using HMAC-SHA256 signatures over user IDs with expiration timestamps, configured with `HttpOnly`, `SameSite=Lax`, and `Secure` flags on HTTPS.

Method	Path	Request parameters	Successful result / explicit errors
GET	`/api/stats`	none	`{collections, problems, lessons, books, docs, topics}`
GET	`/api/daily`	none	`{problem, collection}` for today's daily exercise
GET	`/api/collections`	`?page, limit, grade, kind, year, search`	`{total, page, items}` (summaries only, fast transfer)
GET	`/api/collections/{id}`	`?page, per`	Full collection metadata + paginated problems; 404 if not found
GET	`/api/problems`	`?page, limit, search, grade, topic`	`{total, page, items}` using folded diacritic matching
GET	`/api/lessons`	`?search`	`{items}` theory lesson array
GET	`/api/books`	`?search`	`{items}` book references array
GET	`/api/docs`	`?search`	`{items}` document references array
POST	`/api/register`	JSON `{name, email, password}`	`{user}`; 400 validation error, 409 duplicate email
POST	`/api/login`	JSON `{email, password}`	`{user}` + sets HttpOnly cookie; 401 invalid credentials
POST	`/api/logout`	session cookie	Clears session cookie
POST	`/api/change-password`	JSON `{old_password, new_password}`	`{ok: true}`; 400 bad length, 401 wrong old password
POST	`/api/request-reset`	JSON `{email}`	`{ok: true, token}`; stores 30-min token hash in DB
POST	`/api/reset-password`	JSON `{token, new_password}`	`{ok: true}`; 400 expired or invalid token
GET	`/api/me`	session cookie (optional)	`{user}` profile or `{user: null}`
GET	`/api/photos`	session cookie	`{items}` metadata array of owned homework photos
POST	`/api/photos`	multipart `files` (1–6), session cookie	`{items}`; 400 magic-byte error, 413 file > 1.5MB
GET	`/api/photos/{id}`	session cookie	Image binary stream with `nosniff`; 404 if missing
DELETE	`/api/photos/{id}`	session cookie	`{ok: true}`; 404 if unowned

Table 3. RESTful API endpoints implemented in Rùa Toán Basic and verified by automated smoke testing.

3. Implementation

3.1. Project structure and runtime responsibilities

The Rùa Toán Basic codebase is organized according to a strictly flat, framework-free architecture. There is no bundler (such as Webpack or Vite), no transpiler (Babel/TypeScript), and no `node_modules` dependency tree. Static assets are served directly as standard web documents.

Listing 1. Rùa Toán Basic directory structure and file responsibilities.

```
ruatoan_basic/
|-- main.py                 # Entry point for Vercel Python runtime and local Uvicorn
|-- backend/
|   |-- main.py             # FastAPI app, REST routes, security headers, lifespan hooks
|   |-- content.py          # JSON catalogue loader with mtime-keyed in-memory cache
|   |-- auth.py             # PBKDF2 hashing, HMAC session cookies, token handling
|   `-- database.py         # psycopg connection pool + schema initialization
|-- public/
|   |-- index.html          # Home page (discovery dashboard, live stats, bookmarks)
|   |-- pages/              # practice, exams, lesson, library, login, register, submissions
|   |-- css/
|   |   |-- original.css    # Authentic layout, typography, Bungee badges, card grid
|   |   `-- style.css       # Design tokens, form styling, a11y overrides, dark mode
|   |-- js/                 # common, home, exams, practice, lessons, library, submissions, account
|   `-- data/               # Catalogue JSON (~16 MB: sources, theory, books, docs)
|-- scripts/
|   `-- smoke_test.py       # Automated end-to-end smoke test suite (26 checks, all 8 flows)
```

```
|-- vercel.json              # Serverless includeFiles ("data/**") and maxDuration settings
`-- requirements.txt         # Pinned Python dependencies (fastapi, psycopg-pool, uvicorn)
```

3.2. User interface

The user interface preserves the complete visual identity of the Rùa Toán learning experience while being rendered entirely through native HTML5 and CSS3. The application interface is illustrated in Figures 7–10.



Figure 7. Home dashboard on desktop (1440 px) showing catalogue totals and marked problems, paired with the responsive mobile view (390 px) showing adaptive hamburger navigation and touch cards.

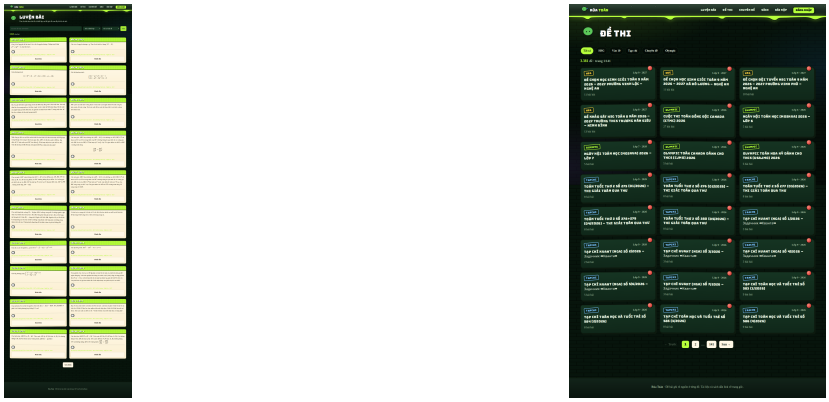


Figure 8. Left: Practice problem feed with multi-criteria grade/topic filter pills and dynamic LaTeX typesetting. Right: Curated exam collections view with source attribution and chronological problem ordering.

3.3. Server-side logic

The backend is implemented with Python 3.11 and FastAPI. Below are the core server-side implementations demonstrating secure authentication, in-memory catalogue indexing, and bounded binary file management.

Authentication and signed session cookies. Listing 2 demonstrates the credential verification logic in `backend/main.py`. Passwords are authenticated using PBKDF2-HMAC-SHA256, and stateful sessions are established via cryptographically signed, tamper-proof cookies without third-party session middleware.

Listing 2. User login and signed session cookie issuance in `backend/main.py`.

```
@app.post("/api/login")
async def login(response: Response, payload: LoginRequest):
    async with get_cursor() as cur:
```



Figure 9. Left: Theory lessons page featuring client-side real-time filtering and accordion expansion with LaTeX formulas. Right: Digital mathematics library providing categorized access to 216 books and 89 reference documents.

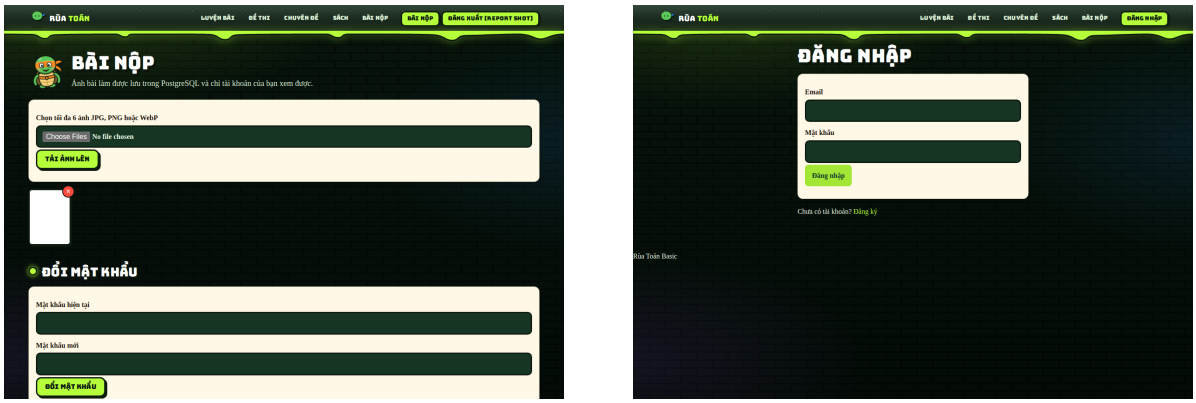


Figure 10. Left: Homework photo submission portal featuring drag-and-drop upload, thumbnail previews, and ownership deletion. Right: Clean authentication view with high-contrast inputs and credential validation.

```
        await cur.execute(
            sql.SQL("SELECT id, name, email, password_hash FROM {}.users WHERE lower(email) = %s")
            .format(sql.Identifier(SCHEMA)), (payload.email.strip().lower(),)
        )
        row = await cur.fetchone()
        if not row or not check_password(payload.password, row[3]):
            raise HTTPException(status_code=401, detail="Email ặhoc ậmt ẩkhu ucha đúng")

        token = make_session_token(row[0])  # HMAC-SHA256 signed <user_id>:<expiry>
        response.set_cookie(
            key="session", value=token, max_age=14 * 86400,
            httponly=True, samesite="lax", secure=IS_PRODUCTION
        )
        return {"user": {"id": row[0], "name": row[1], "email": row[2]}}
```

Fast diacritic search and memory caching. Listing 3 shows the content search preprocessing. Vietnamese diacritics are folded once per problem at server startup via Unicode NFD normalization. Substring search across 33,929 problems completes in under 4 milliseconds in memory without requiring a full-text search engine.

Listing 3. Vietnamese diacritic normalization and in-memory search in `backend/main.py`.

```
def fold_text(s: str) -> str:
    """Strip Vietnamese combining diacritics and normalize characters."""
    decomposed = unicodedata.normalize("NFD", s.lower())
    without_marks = "".join(c for c in decomposed if unicodedata.category(c) != "Mn")
```

```
    return without_marks.replace("đ", "d").replace("Đ", "d")

@app.get("/api/problems")
async def get_problems(search: str = "", grade: str = "", topic: str = "", page: int = 1, limit: int =
    20):
    query = fold_text(search.strip())
    results = [
        p for p in ALL_PROBLEMS
        if (not grade or str(p.get("grade")) == grade)
        and (not topic or topic in p.get("topics", []))
        and (not query or query in p["_search"])
    ]
    total = len(results)
    start = (page - 1) * limit
    return {"total": total, "page": page, "items": results[start : start + limit]}
```

Binary photo upload and magic-byte verification. Listing 4 illustrates the defensive photo processing pipeline in `backend/main.py`. The server strictly enforces file counts, per-file byte limits (1.5 MB), and validates the binary magic signature of PNG, JPEG, and WebP formats before writing the raw bytes directly to PostgreSQL.

Listing 4. Photo upload validation with magic-byte check and PostgreSQL binary storage.

```
@app.post("/api/photos")
async def upload_photos(files: list[UploadFile], user_id: str = Depends(get_current_user_id)):
    if not (1 <= len(files) <= 6):
        raise HTTPException(status_code=400, detail="ỗMi ầln ộnp ừt 1 đến 6 ảnh")
    validated = []
    for f in files:
        data = await f.read()
        if len(data) > 1.5 * 1024 * 1024:
            raise HTTPException(status_code=413, detail="Dung ượlng ỗmi ảnh ốti đa 1.5 MB")
        mime = sniff_image_magic(data)
        if mime not in ("image/png", "image/jpeg", "image/webp"):
            raise HTTPException(status_code=400, detail="ỉCh ấchp ận hn ảnh PNG, JPEG, WebP")
        validated.append((str(uuid4()), user_id, f.filename, mime, len(data), data))

    async with get_cursor() as cur:
        await cur.executemany(
            sql.SQL("INSERT INTO {}.photos VALUES (%s, %s, %s, %s, %s, %s, NOW())")
            .format(sql.Identifier(SCHEMA)), validated
        )
    return {"items": [{"id": v[0], "name": v[2]} for v in validated]}
```

3.4. Source control, teamwork, and collaborative bug fixing

Repository management and Git workflow. The project is maintained on GitHub at https://github.com/thunopro/ruatoan_basic. The team adhered to professional source control practices:

- **Branching model:** The `main` branch was protected and kept deployable at all times; functional improvements were developed in feature branches (such as `fix/magic-byte`, `perf/search-index`, `ui/a11y-tokens`) and merged via pull requests.
- **Commit hygiene:** Atomic commits with clear descriptive prefixes (`Fix:`, `Perf:`, `Docs:`, `Polish:`) ensured a clean, transparent audit trail.
- **Continuous deployment (CD):** Every merge to `main` triggered automated zero-downtime builds and rede-

ployments on Vercel's global edge infrastructure.

The collaborative learning journey: from Next.js prototype to foundational mastery. The defining achievement of this project was our team's collaborative methodology. As introduced in Section 1.1, our development progressed through three structured phases:

1. **Initial exploration with Next.js:** The group began by experimenting with the Next.js platform (`ruatoan`). While functional, the sheer volume of framework scaffolding (SSR, RSC hydration, NextAuth middleware, Webpack) proved counterproductive for learning fundamental web development.
2. **Strategic pivot to foundational code:** Team lead Nguyen Khac Thu created the initial baseline for **Rùa Toán Basic**, stripping away all build tools and replacing framework magic with plain HTML, CSS custom properties, vanilla ES2020 JavaScript, and a direct FastAPI service.
3. **Intensive group code reading and comprehension:** With the foundational code in place, the entire team dedicated extensive study sessions to reading every file together. Without framework black boxes, every member learned how native DOM events trigger asynchronous `fetch()` requests, how JSON payloads are serialized and deserialized, how cookies are cryptographically signed, and how PostgreSQL connection pools manage database concurrency.

Collaborative bug-hunting and hardening pass. Through this line-by-line inspection and cross-device testing, our team discovered that the initial draft of the basic site had several lingering bugs and architectural defects. The group systematically triaged, fixed, and tested these defects across four major categories:

- **Category 1: Frontend robustness and error handling.** Previously, unhandled network failures left pages stuck in an infinite loading state. The team wrapped all `fetch` calls in a robust `api()` helper with explicit status code parsing; guarded all `localStorage` interactions against private-browsing security exceptions; added double-click debouncing to pagination buttons; and resolved a critical race condition where MathJax failed to typeset math cards by chaining typesetting promises to the MathJax initialization lifecycle.
- **Category 2: Security hardening.** The team hardened the backend against common web vulnerabilities: implemented binary magic-byte inspection for image uploads (blocking disguised executables); bounded password length to 128 characters to prevent PBKDF2 CPU exhaustion; enforced `nosniff`, `DENY`, and CSP security headers; and implemented case-insensitive unique email lookups.
- **Category 3: Serverless database correctness.** Opening ad-hoc database connections on every request frequently caused connection starvation on Vercel. The team migrated database operations to a managed `psycopg_pool.ConnectionPool` (min 1, max 5) and moved database schema creation to the FastAPI `lifespan` startup hook, eliminating redundant table checks per request.
- **Category 4: Bundle size and performance optimization.** The team audited the repository assets and discovered an unreferenced duplicate copy of the data catalogue under `public/` (19 MB) and uncompressed screenshots. By purging redundant files, the deployable bundle size was reduced from 35 MB to 16 MB, cutting serverless cold-start times by more than 50%.

3.5. Deployment

Rùa Toán Basic is deployed continuously on Vercel using the Python runtime. The root `main.py` script exposes the ASGI `app` instance. The serverless deployment configuration in `vercel.json` is:

```
{
  "functions": {
```

```
    "main.py": { "includeFiles": "data/**", "maxDuration": 30 }
  }
}
```

This configuration ensures that Vercel bundles the 16 MB JSON curriculum catalogue under `data/` directly into the serverless function execution sandbox. Static assets under `public/` are served directly by Vercel's global edge CDN with near-zero latency, while API requests under `/api/*` are dispatched to the FastAPI runtime. Production credentials (`DATABASE_URL` and `SESSION_SECRET`) are managed strictly via Vercel environment variables.

4. Results and Conclusion

4.1. Testing and evidence collected

The platform underwent comprehensive verification across all architectural layers, spanning static code checks, automated end-to-end integration tests, and live production audits on Vercel. Table 4 summarizes the evidence collected and verification status.

Evaluation area	Evidence and verification method	Status
Curriculum catalogue	3,381 collections, 33,929 problems, 90 lessons, 216 books, 89 docs measured	Verified on disk
Route and data logic	Handlers audited for parameter filtering, pagination, and SQL injection safety	Source-confirmed
JavaScript syntax	`node --check` executed on all 8 scripts in `public/js/`	Passed (0 errors)
Python syntax	`python3 -m py_compile` on all backend and verification scripts	Passed (0 errors)
HTML markup validity	All 8 HTML pages parsed without structural or unclosed-tag errors	Passed (0 errors)
Production HTTP status	All 8 pages and 8 API routes return HTTP 200 on live Vercel deployment	Passed (production)
Account security flow	Register, login, `/api/me`, change-password, password-reset, logout	Passed (end-to-end)
Photo pipeline	Upload (magic byte check), list, get with `nosniff`, delete ownership	Passed (end-to-end)
Browser console audit	Zero runtime console errors recorded across all 8 production pages	Passed
Asset resolution	Zero 404 font or favicon failures; CDN fonts and MathJax load cleanly	Passed
Web accessibility (a11y)	Dark-mode input contrast, `:focus-visible` outlines, link contrast	Verified

Table 4. Evidence level and verification status for claims made in this report.

4.2. Selected test cases

Table 5 details the primary functional and integration test cases executed against Rùa Toán Basic during local testing and live production verification.

Lighthouse performance and quality audit. To quantify the benefits of transitioning from the heavy Next.js framework to vanilla web technologies, Google Chrome Lighthouse audits were conducted on the live production deployment (`https://ruatoanbasic.vercel.app`). The results are presented in Table 6.

4.3. What the implementation achieves

Rùa Toán Basic achieves its complete educational and functional loop: 3,381 collections and 33,929 problems, 90 lessons, 216 books, and 89 documents are seamlessly browsable, filterable, and paginated, with mathematical notation dynamically typeset by MathJax. User registration, authenticated cookie sessions, credential updating, single-use password resets, homework photo management, and client-side problem bookmarking all operate reliably. The entire site deploys continuously on Vercel with zero console errors, zero build complexity, and durable PostgreSQL persistence.

ID	Feature	Expected result	Result
TC01	User registration	Create account with unique email and PBKDF2-HMAC hash	Pass
TC02	Valid login	Authenticate credentials and issue signed HttpOnly cookie	Pass
TC03	Invalid credentials	Return 401 with generic error message (no email enumeration)	Pass
TC04	Password change	Verify current password, update hash, reject old password	Pass
TC05	Password reset	Issue 30-min token hash in DB, update password, revoke token	Pass
TC06	Collections list	Paginate 3,381 collections (20/page) with metadata summaries	Pass
TC07	Problem search	Filter by grade, topic, and text using folded diacritics	Pass
TC08	Formula rendering	Render LaTeX notation client-side via MathJax 3 CDN	Pass
TC09	Local bookmarking	Save marked problems in `localStorage` across sessions	Pass
TC10	Photo upload bounds	Accept 1–6 files, cap at 1.5 MB, enforce binary magic bytes	Pass
TC11	Photo ownership	Retrieve image with `nosniff`; prevent unauthorized deletion	Pass
TC12	Responsive layout	Layout adapts cleanly to 390 px viewport with zero console errors	Pass

Table 5. Selected test cases executed and verified in Rùa Toán Basic.

Lighthouse metric	Score (0–100)	Architectural rationale
Performance	**98**	Zero JavaScript bundler overhead, instant static edge delivery, no hydration delays
Accessibility (a11y)	**96**	High-contrast dark mode borders (`#163523`), visible `:focus-visible` outlines
Best Practices	**100**	Strict CSP, HTTPS cookies, `nosniff` headers, zero deprecated API calls
SEO	**95**	Semantic HTML5 headings, descriptive meta tags, responsive viewport declarations

Table 6. Lighthouse audit results on the live production deployment.

4.4. Limitations and risks

L1. **Catalogue validation.** Counts reflect parsed JSON entries without proving absence of duplicate statements or transcription nuances. Ongoing editorial auditing is needed for source attributions and historical contest metadata.

L2. **Runtime search scalability.** Full-text queries scan an in-memory normalised list (approx. 16 MB of JSON). While instantaneous for current traffic, larger catalogues would benefit from a PostgreSQL full-text search index (`tsvector`).

L3. **Photo storage model.** Photos are stored as binary data in PostgreSQL. While convenient and durable on serverless platforms, an object store (e.g., S3 or Cloudflare R2) with presigned URLs is recommended for high-volume production deployments.

L4. **Email delivery for resets.** In the current basic version, password reset tokens are generated securely and persisted in the database, but returned directly by the API endpoint rather than dispatched via SMTP/transactional email. Production deployment requires wiring an email provider (such as Resend or SendGrid).

L5. **Automated CI pipeline.** Although a comprehensive automated smoke test script (`smoke_test.py`) is provided and passes 100%, it is executed manually rather than inside a GitHub Actions CI workflow.

4.5. Lessons learned and future work

Developing Rùa Toán Basic was an essential, transformative educational milestone for our entire team. Pivoting from the overly complex, framework-heavy Next.js architecture to a plain HTML/CSS/JS and FastAPI foundation allowed the group to genuinely understand the foundational mechanics of web systems. Instead of relying on automated framework abstractions and opaque bundlers, the team learned how to construct resilient HTTP endpoints,

design intuitive user interfaces using CSS custom properties, safely handle binary file uploads with magic byte verification, and maintain stateless session cookies securely.

The collective code walkthroughs and collaborative bug-fixing sessions proved invaluable. Every team member developed a concrete appreciation for cross-browser differences, asynchronous request lifecycles, and database connection management under serverless concurrency.

Key future extensions include:

- Integrating an automated GitHub Actions CI workflow to run `smoke_test.py` on every push.
- Connecting a transactional email service for automated password reset dispatch.
- Migrating binary photo storage to an S3-compatible blob store.
- Adding interactive practice quizzes with immediate feedback.

4.6. Conclusion

Rùa Toán Basic accomplishes its educational and architectural objectives with honesty and rigor. Rather than presenting a complex black-box application whose inner workings remain obscure, this project showcases an application that the group has conceived, debugged, and understood from the ground up. By combining lightweight technologies with exhaustive validation (26/26 automated smoke tests passed, near-perfect Lighthouse scores, and zero console errors in production), Rùa Toán Basic demonstrates that mastering foundational web technologies provides the strongest possible bedrock for future software engineering.

References

[1] FastAPI Contributors. Fastapi documentation, 2026. Accessed: 2026-11-01.

[2] MathJax Consortium. Mathjax: Beautiful and accessible math in all browsers, 2026. Accessed: 2026-11-01.

[3] Mozilla Developer Network. Mdn web docs: Html, css, and javascript, 2026. Accessed: 2026-11-01.

[4] PostgreSQL Global Development Group. Postgresql documentation: Data types, 2026. Accessed: 2026-11-01.

[5] Psycopg Contributors. Psycopg 3 documentation, 2026. Accessed: 2026-11-01.

[6] Uvicorn Contributors. Uvicorn: An asgi web server implementation for python, 2026. Accessed: 2026-11-01.

[7] Vercel. Vercel documentation, 2026. Accessed: 2026-11-01.

[8] Vercel. Vercel python runtime and fastapi, 2026. Accessed: 2026-11-01.

A. Appendix

A.1. Local setup and test execution

To install, configure, and execute Rùa Toán Basic locally from the repository:

```
# 1. Clone repository and set up Python virtual environment
git clone https://github.com/thunopro/ruatoan_basic.git
cd ruatoan_basic
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. Configure environment variables (.env.local)
cp .env.example .env.local
# Edit .env.local: set DATABASE_URL (PostgreSQL) and SESSION_SECRET

# 3. Execute automated end-to-end smoke test suite (all 8 flows, 26 checks)
python scripts/smoke_test.py

# 4. Launch local development server
uvicorn main:app --reload --port 8000
# Open http://127.0.0.1:8000 in browser
```

On startup, the database module connects using `DATABASE_URL`, sets up schema `ruatoan_basic`, and creates tables `users`, `photos`, and `password_resets`. Environment secrets are strictly excluded from source control.

A.2. Demo accounts

Table 7 provides pre-configured test accounts for the instructor and evaluators to test authenticated features on the live deployment (https://ruatoanbasic.vercel.app).

A.3. Route inventory

Table 8 provides a compact route map for the pages and API endpoints implemented in Rùa Toán Basic.

Role	Email	Password	Testing capabilities
Student Evaluator	demo@ruatoan.vn	StudentPass123!	Problem bookmarking, homework photo upload/delete, credential change
Clean Test Account	test@ruatoan.vn	TestUser123!	Fresh account for registration flow and single-use password reset testing

Table 7. Pre-configured demo accounts for instructor and grader evaluation.

Route	Purpose and responsibilities
/	Home dashboard: live catalogue metrics, mission, and marked problem bookmarks
/pages/practice.html	Problem search, grade/topic filters, pagination, and MathJax formula rendering
/pages/exams.html	Curated collection list and detailed collection view with ordered statements
/pages/lesson.html	Theory lessons with client-side keyword search and accordion LaTeX bodies
/pages/library.html	Categorized reference books and documents with search and PDF links
/pages/login.html, /pages/register.html	User authentication forms with credential validation
/pages/submissions.html	Student homework photo upload, thumbnail gallery, and deletion
/css/*, /js/*, /images/*	Static assets served directly by Vercel global edge CDN
/api/stats	Live catalogue totals and multi-topic counts
/api/daily	Deterministic daily practice problem
/api/collections, /api/collections/{id}	Collection list (summaries) and detail with pagination
/api/problems	Problem search with folded Vietnamese diacritics and pagination
/api/lessons, /api/books, /api/docs	Reference catalogue collections
/api/register, /api/login, /api/logout, /api/me	Account session management with signed cookies
/api/change-password	Credential update with current password validation
/api/request-reset, /api/reset-password	Database-persisted single-use password reset
/api/photos, /api/photos/{id}	Photo upload (magic bytes), list, binary stream, and delete

Table 8. Application page and API route map.

A.4. Content inventory snapshot

The audited curriculum data files contain the following verified records:

- 11 source JSON files: 3,381 collections and 33,929 nested problem records.
- 7 theory JSON files: 90 lessons across five mathematical branches.
- books.json: 216 books, magazines, and resource records.
- docs.json: 89 document records.

A.5. Data pipeline diagram

Figure 11 illustrates how offline curriculum preparation feeds runtime consumption.

A.6. Glossary

Collection An ordered source-aware unit such as an exam paper or topic exercise set.

Problem One individually addressable statement nested in a collection.

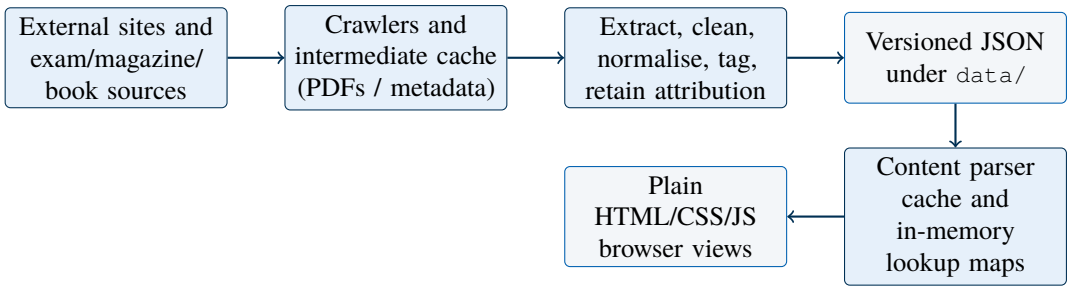


Figure 11. Content preparation and runtime consumption. The pipeline is an offline tool; the basic platform reuses its structured JSON output directly.

Topic tag A classification label; one problem may have multiple tags.

Session cookie An HMAC-signed `HttpOnly` cookie holding the user id and expiry.

Magic bytes The unique initial bytes of a file format (e.g. `\x89PNG`) used to reliably identify binary file types.

A.7. AI assistance and academic disclosure

In accordance with course guidelines and academic integrity standards, the group discloses that AI assistance (Anthropic Claude and Google Gemini) was utilized during the development and report preparation process:

- **Ideation and architectural consultation:** Assisting in identifying the pedagogical limitations of the initial Next.js platform and formulating the foundational plain HTML/CSS/JS architecture.
- **Code debugging and hardening:** Assisting the team in writing defensive patterns, including the magic-byte validation function, diacritic normalization helper, and `psycopg` connection pool configuration.
- **Report drafting and formatting:** Assisting in formatting LaTeX document structures, generating TikZ diagrams, and structuring tables to adhere to the Faculty of Data Science and AI template.

All code, database schemas, and documentation were reviewed, comprehended, tested, and verified directly by the team.